MLA0303_RL_PROGRAM_2 — Program with Output

Python Source Code

```
"""
Experiment 2: Reinforcement Learning agent for a Smart Home Robot
Tabular Q-Learning is used so the robot learns, purely through trial and
error interaction with the environment, how to navigate to its charging
dock while avoiding furniture obstacles.
"""

import random

# ---------------------------------------------------------------------
# 1. Smart home grid environment
# ---------------------------------------------------------------------
GRID_SIZE = 5
OBSTACLES = {(1, 1), (1, 2), (3, 3), (2, 0)}   # furniture positions
DOCK = (4, 4)                                   # charging dock (goal)

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}

ALPHA = 0.1        # learning rate
GAMMA = 0.95        # discount factor
EPSILON_START = 1.0
EPSILON_MIN = 0.05
EPSILON_DECAY = 0.995
EPISODES = 500
MAX_STEPS = 100


def valid_cells():
    return [(r, c) for r in range(GRID_SIZE) for c in range(GRID_SIZE) if (r, c) not in OBSTACLES]


def step(state, action):
    dr, dc = DELTA[action]
    nxt = (state[0] + dr, state[1] + dc)
    if not (0 <= nxt[0] < GRID_SIZE and 0 <= nxt[1] < GRID_SIZE) or nxt in OBSTACLES:
        return state, -5, False          # bumped into wall / furniture
    if nxt == DOCK:
        return nxt, 20, True             # reached charging dock
    return nxt, -1, False                # normal move (step cost)


# ---------------------------------------------------------------------
# 2. Q-Learning
# ---------------------------------------------------------------------
Q = {(s, a): 0.0 for s in valid_cells() for a in ACTIONS}


def choose_action(state, epsilon):
    if random.random() < epsilon:
        return random.choice(ACTIONS)
    q_values = {a: Q[(state, a)] for a in ACTIONS}
    return max(q_values, key=q_values.get)


def train():
    epsilon = EPSILON_START
    rewards_per_episode = []
    starts = [c for c in valid_cells() if c != DOCK]
    for ep in range(EPISODES):
        state = random.choice(starts)
        total_reward = 0
        for _ in range(MAX_STEPS):
            action = choose_action(state, epsilon)
            next_state, reward, done = step(state, action)
            best_next = max(Q[(next_state, a)] for a in ACTIONS)
            Q[(state, action)] += ALPHA * (reward + GAMMA * best_next - Q[(state, action)])
            state = next_state
            total_reward += reward
            if done:
                break
        rewards_per_episode.append(total_reward)
        epsilon = max(EPSILON_MIN, epsilon * EPSILON_DECAY)
    return rewards_per_episode


# ---------------------------------------------------------------------
# 3. Evaluation
```

```
# ----------------------------------------------------------------------
def navigate(start):
    state = start
    path = [state]
    for _ in range(MAX_STEPS):
        action = choose_action(state, epsilon=0.0)   # greedy
        state, _, done = step(state, action)
        path.append(state)
        if done:
            break
    return path


if __name__ == "__main__":
    random.seed(3)
    rewards = train()
    print(f"Average reward, first 10 episodes : {sum(rewards[:10]) / 10:.2f}")
    print(f"Average reward, last 10 episodes  : {sum(rewards[-10:]) / 10:.2f}\n")

    for start in [(0, 0), (4, 0), (0, 4)]:
        path = navigate(start)
        print(f"Start {start} -> Path: {path}")
```

Execution Output

```
Average reward, first 10 episodes : -124.20
Average reward, last 10 episodes  : 16.70

Start (0, 0) -> Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)]
Start (4, 0) -> Path: [(4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Start (0, 4) -> Path: [(0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]

STDERR:
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 772, in warm_spreadsheet_
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/connection.py", line 37, in get_or_create_client
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/daemon.py", line 124, in start_daemon
TimeoutError: Timed out waiting for artifact tool daemon socket. Set ARTIFACT_TOOL_RPC_DAEMON_STARTUP_TIMEOUT_S=<seconds
```